

项目什么场景使用线程池？参数如何设置？

什么是线程池

线程池是一种基于池化思想管理线程的工具，使用线程池可以减少创建销毁线程的开销，避免线程过多导致系统资源耗尽。在高并发以及大批量的任务处理场景，线程池的使用是必不可少的。

线程过多会带来额外的开销，其中包括创建销毁线程的开销、调度线程的开销等等，同时也降低了计算机的整体性能。线程池维护多个线程，等待监督管理者分配可并发执行的任务。这种做法，一方面避免了处理任务时创建销毁线程开销的代价，另一方面避免了线程数量膨胀导致的过分调度问题，保证了对内核的充分利用。

什么场景下用线程池

线程池是 Java 中用于管理和重用线程的一种机制，它适用于各种多线程的应用场景。以下是一些适合使用线程池的场景：

1. 异步任务处理：当需要执行一些异步任务，但又不希望为每个任务都创建一个新线程时，可以使用线程池。例如，处理用户提交的后台任务、异步日志记录等。
2. 服务器端应用程序：在服务器端开发中，通常需要同时处理多个客户端请求。线程池可以管理这些客户端请求的处理，以避免为每个请求创建新线程，从而提高性能和资源利用率。
3. 并发编程：在多线程编程中，如果需要控制并发度，限制同时运行的线程数量，线程池是一个有用的工具。它可以避免创建太多线程，导致资源竞争和性能问题。
4. 定时任务：JDK 线程池还支持定时执行任务。可以使用 `ScheduledExecutorService` 接口来执行按计划运行的任务，例如定时器、定期数据备份等。
5. 资源管理：在一些需要管理有限资源的情况下，线程池可以帮助控制资源的分配。例如，数据库连接池通常使用线程池来管理数据库连接。
6. 避免线程创建和销毁的开销：创建和销毁线程是昂贵的操作，线程池可以重用线程，减少这些开销，提高性能。

JDK 线程池适用于各种需要管理和重用线程的场景，可以提高多线程应用程序的性能和可维护性。但需要根据具体的应用需求和性能要求来选择不同类型的线程池，并合理配置线程池的参数。

12306 如何使用线程池

12306 中有个很好的场景，如果一个用户购买多种类型的车座，比如同一订单购买了一等座和二等座，那么就要按照不同的座位逻辑进行选座。

上述这个流程在我们系统里是串行的，也就是说执行完一等座选座后再执行二等座。这个逻辑是可以并行的，可以极大程度上提高咱们的执行效率，提升系统吞吐量。

原始座位分配逻辑

根据上文描述，第一版座位分配场景通过串行的方式分配座位。如果遇到一个用户购买了两到三个座位类型，性能执行稍差。

```
// 根据用户购买的座位类型进行分组
Map<Integer, List<PurchaseTicketPassengerDetailDTO>> seatTypeMap =
passengerDetails.stream()
    .collect(Collectors.groupingBy(PurchaseTicketPassengerDetailDTO::getSeatType));
// 分配好用户相关的座位信息
List<TrainPurchaseTicketRespDTO> actualResult = new ArrayList<>();
// 串行化执行座位分配流程，比如先执行一等座，再执行二等座...
seatTypeMap.forEach((seatType, passengerSeatDetails) -> {
    SelectSeatDTO selectSeatDTO = SelectSeatDTO.builder()
        .seatType(seatType)
        .passengerSeatDetails(passengerSeatDetails)
        .requestParam(requestParam)
        .build();
    // 座位分配
    List<TrainPurchaseTicketRespDTO> aggregationResult =
abstractStrategyChoose.chooseAndExecuteResp(VehicleTypeEnum.findNameByCode(trainType) +
VehicleSeatTypeEnum.findNameByCode(seatType), selectSeatDTO);
    if (!Objects.equals(aggregationResult.size(), passengerSeatDetails.size())) {
        throw new ServiceException("站点余票不足，请尝试更换座位类型或选择其它站点");
    }
    // 分配好座位的用户信息添加到返回集合中
    actualResult.addAll(aggregationResult);
});
```

代入线程池场景

第二个版本，我们引入线程池去做这个单订单多座位购票场景。

首先引入动态可监控线程池 [Hippo4j](#)，通过中间件定义的增强后的动态线程池帮助我们完成并行业务。

动态线程池参数的定义我们放到下面章节讲解。

```
private final ThreadPoolExecutor selectSeatThreadPoolExecutor;

// 根据用户购买的座位类型进行分组
Map<Integer, List<PurchaseTicketPassengerDetailDTO>> seatTypeMap =
passengerDetails.stream()
    .collect(Collectors.groupingBy(PurchaseTicketPassengerDetailDTO::getSeatType));
// 分配好用户相关的座位信息
List<TrainPurchaseTicketRespDTO> actualResult = new ArrayList<>();
// 串行化执行座位分配流程，比如先执行一等座，再执行二等座...
seatTypeMap.forEach((seatType, passengerSeatDetails) -> {
    selectSeatThreadPoolExecutor.execute(() -> {
        SelectSeatDTO selectSeatDTO = SelectSeatDTO.builder()
            .seatType(seatType)
            .passengerSeatDetails(passengerSeatDetails)
            .requestParam(requestParam)
            .build();
        // 执行策略模式分配座位
        List<TrainPurchaseTicketRespDTO> aggregationResult =
abstractStrategyChoose.chooseAndExecuteResp(VehicleTypeEnum.findNameByCode(trainType) +
VehicleSeatTypeEnum.findNameByCode(seatType), selectSeatDTO);
        if (!Objects.equals(aggregationResult.size(), passengerSeatDetails.size())) {
            throw new ServiceException("站点余票不足，请尝试更换座位类型或选择其它站点");
        }
        // 分配好座位的用户信息添加到返回集合中
        actualResult.addAll(aggregationResult);
    });
});
```

线程池执行方案

第二种解决方案看着是挺好的，但是还有一些并行以及性能的问题，如下：

1. 如果用户下单仅选择单一座位，通过线程池执行会额外增加性能消耗。
2. 线程池并行的情况下，向 actualResult 添加数据，会存在并发问题。因为 actualResult 是 ArrayList。

结合上面两个问题，对代码做了相关优化。最终代码如下所示：

```
private final ThreadPoolExecutor selectSeatThreadPoolExecutor;

// 根据用户购买的座位类型进行分组
Map<Integer, List<PurchaseTicketPassengerDetailDTO>> seatTypeMap =
passengerDetails.stream()
    .collect(Collectors.groupingBy(PurchaseTicketPassengerDetailDTO::getSeatType));
// 分配好用户相关的座位信息，容器修改为线程安全的数组集合
List<TrainPurchaseTicketRespDTO> actualResult = Collections.synchronizedList(new
ArrayList<>());
// 如果座位类型超过单个并行分配座位
if (seatTypeMap.size() > 1) {
    // 串行化执行座位分配流程，比如先执行一等座，再执行二等座...
    seatTypeMap.forEach((seatType, passengerSeatDetails) -> {
        selectSeatThreadPoolExecutor.execute(() -> {
            SelectSeatDTO selectSeatDTO = SelectSeatDTO.builder()
                .seatType(seatType)
                .passengerSeatDetails(passengerSeatDetails)
                .requestParam(requestParam)
                .build();
            // 执行策略模式分配座位
            List<TrainPurchaseTicketRespDTO> aggregationResult =
abstractStrategyChoose.chooseAndExecuteResp(VehicleTypeEnum.findNameByCode(trainType) +
VehicleSeatTypeEnum.findNameByCode(seatType), selectSeatDTO);
            if (!Objects.equals(aggregationResult.size(), passengerSeatDetails.size())) {
                throw new ServiceException("站点余票不足，请尝试更换座位类型或选择其它站
点");
            }
            // 分配好座位的用户信息添加到返回集合中
            actualResult.addAll(aggregationResult);
        });
    });
}
```

```

    });
} else {
    // 如果座位类型只有单个，串行执行即可
    seatTypeMap.forEach((seatType, passengerSeatDetails) -> {
        List<TrainPurchaseTicketRespDTO> aggregationResult = distributeSeats(trainType,
seatType, requestParam, passengerSeatDetails);
        actualResult.addAll(aggregationResult);
    });
}

```

最终解决方案

上面有个问题是线程池执行是异步的，有可能我们购票流程都结束了，这里还没执行完分配座位逻辑。

为此，我们需要拿到线程池执行的结果后再继续执行后续的提交订单逻辑。

```

Map<Integer, List<PurchaseTicketPassengerDetailDTO>> seatTypeMap =
passengerDetails.stream()
    .collect(Collectors.groupingBy(PurchaseTicketPassengerDetailDTO::getSeatType));
List<TrainPurchaseTicketRespDTO> actualResult = new CopyOnWriteArrayList<>();
if (seatTypeMap.size() > 1) {
    // 添加多线程执行结果类获取 Future，流程执行完成后获取
    List<Future<List<TrainPurchaseTicketRespDTO>>> futureResults = new ArrayList<>();
    seatTypeMap.forEach((seatType, passengerSeatDetails) -> {
        Future<List<TrainPurchaseTicketRespDTO>> completableFuture =
selectSeatThreadPoolExecutor
        .submit(() -> distributeSeats(trainType, seatType, requestParam,
passengerSeatDetails));
        futureResults.add(completableFuture);
    });
    // 获取座位分配结果
    futureResults.parallelStream().forEach(completableFuture -> {
        try {
            actualResult.addAll(completableFuture.get());
        } catch (Exception e) {
            throw new ServiceException("站点余票不足，请尝试更换座位类型或选择其它站点");
        }
    });
} else {

```

```
seatTypeMap.forEach((seatType, passengerSeatDetails) -> {  
    List<TrainPurchaseTicketRespDTO> aggregationResult = distributeSeats(trainType,  
seatType, requestParam, passengerSeatDetails);  
    actualResult.addAll(aggregationResult);  
});  
}
```

为了避免座位分配数量出现问题，我们应该在座位全部分配完成后进行一次验证。

```
if (CollUtil.isEmpty(actualResult) || !Objects.equals(actualResult.size(),  
passengerDetails.size())) {  
    throw new ServiceException("站点余票不足，请尝试更换座位类型或选择其它站点");  
}
```

12306 线程池参数如何设置

有问题的线程池参数

思考一个问题，如果咱们创建一个类似于这种线程池，业务使用会不会有问题？

```
ThreadPoolExecutor selectSeatThreadPoolExecutor = new ThreadPoolExecutor(  
    24, // 核心线程数  
    36, // 最大线程数  
    20L, // keepAliveTime  
    TimeUnit.SECONDS, // keepAliveTime 时间单位  
    new LinkedBlockingQueue<>(4096), // 阻塞队列  
    new ThreadPoolExecutor.AbortPolicy() // 拒绝策略：抛出异常  
);
```

我们先不说核心线程数和最大线程数是否合理，这个不在本次文章讨论范围内。线程池核心线程数和最大线程数参考文章：[线程池参数怎么配置比较合理？](#)

本次会从阻塞队列以及拒绝策略的方式进行解析。阻塞队列设置 `LinkedBlockingQueue`，容量设置为 4096 有什么问题？拒绝策略设置 `AbortPolicy` 有什么问题？

线程池线程数是有限的，如果大量任务堆积在阻塞队列，这对于业务的执行是有比较大的延迟。

我们期望的结果是，线程池不堆积任务，如果线程池的线程满了就让提交任务的主线程（Web容器执行线程）执行分配座位逻辑。简单来说，就是线程池线程都在执行任务时，我们将并行的逻辑降级为串行。

重构后的线程池参数

为了实现上述业务逻辑，我们通过 Hippo4j 修改创建线程池的阻塞队列和拒绝策略。

将阻塞队列设置为 SynchronousQueue，一个没有容量的阻塞队列，没有容量自然也不会有堆积。

如果线程池满了会触发拒绝策略，如果抛异常就违反了咱们设计的初衷。从 AbortPolicy 修改为 CallerRunsPolicy，代表如果触发拒绝策略，则由提交任务的线程运行该任务。

```
@Bean
@DynamicThreadPool
public ThreadPoolExecutor selectSeatThreadPoolExecutor() {
    String threadPoolId = "select-seat-thread-pool-executor";
    return ThreadPoolBuilder.builder()
        .threadPoolId(threadPoolId)
        .threadFactory(threadPoolId)
        .workQueue(BlockingQueueTypeEnum.SYNCHRONOUS_QUEUE)
        .corePoolSize(24)
        .maximumPoolSize(36)
        .allowCoreThreadTimeOut(true)
        .keepAliveTime(60, TimeUnit.MINUTES)
        .rejected(new ThreadPoolExecutor.CallerRunsPolicy())
        .dynamicPool()
        .build();
}
```

文末总结

通过线程池方式改造用户选择多个列车座位需求，从最初的串行修改为并行，一定程度上提升了接口的响应时间。同时，如果用户没有选择多个列车座位，依旧按照原有流程执行，并不会带来额外的消耗。

最终在选择线程池参数时，给大家做了一个根据业务特点而形成的线程池配置。这也是我一直想和大家说的，没有一成不变的线程池参数，只有根据合适的业务场景不断假设和调优，最终演变成合适的线程池。

更新: 2024-04-06 16:45:17

原文: <<https://www.yuque.com/magestack/12306/cwh86n5b149dz91a>>